

Engineering Brief: FRS Field Detection Interface (v1)

1. Background

Field Resonance Segmentation (FRS) is a research framework for detecting affective-semantic patterns in dialogue. It grew out of professional work in psychographic audience segmentation and proposes that conversation segments occupy a small number of recurring "field" states — working hypothesis: five — that should be detectable as cluster structure in embedding space, without supervision.

This project is **not** a request to test or validate that hypothesis. It is a request to build the **instrument**: an interactive analysis tool that lets the researcher run transcripts through an embedding → reduction → clustering pipeline, explore the results visually, and label what she finds. All interpretive work remains with the researcher.

The researcher is not a programmer. The finished tool must be operable end-to-end without writing code, and documentation must be in plain English.

2. Objective

A local, interactive application ("the workbench") that:

1. Ingests dialogue transcripts from two source types: the researcher's own exported chat transcripts, and at least one public dialogue corpus.
2. Segments, embeds, reduces, and clusters the dialogue with adjustable settings.
3. Displays results as an explorable 2D map where every point can be read as text.
4. Lets the researcher tag clusters and segments with her own labels and export those labels.
5. Builds in **verification controls** (Section 5) so that apparent structure can be distinguished from artifacts of the method.

3. Core pipeline (v1 scope)

****Ingest****

- Accept plain-text and JSON chat transcripts (formats to be agreed at kickoff — researcher will supply samples).
- Loader for the ****AMI Meeting Corpus**** (public, collaborative design-team meetings). A second public corpus (e.g. CANDOR) is a nice-to-have, not required.
- All processing must run ****locally****. The researcher's transcripts are personal data and must not be sent to third-party APIs without her explicit sign-off. If a hosted embedding model is proposed, this must be flagged and agreed first.

****Segmentation****

- Configurable unit of analysis: single turn / sliding window of N turns / manually bounded episode. Sensible defaults proposed by the engineer.

****Embedding****

- Sentence-embedding model of the engineer's recommendation (open-weights preferred for local processing). Choice and rationale documented in plain English.

****Dimensionality reduction****

- UMAP with adjustable key parameters, exposed in the interface with brief plain-English tooltips (e.g. "smaller values = tighter local detail").

****Clustering****

- Two interchangeable methods: one with a user-set number of clusters (e.g. k-means) and one density-based that discovers cluster count (e.g. HDBSCAN). The researcher must be able to switch between them and adjust k freely.

****Visualisation & exploration****

- Interactive 2D scatter: hover/click any point to read the underlying segment text and metadata (source, speaker, position in conversation).
- Colour by cluster; filter by source corpus, conversation, or speaker.
- Side panel listing representative exemplar segments per cluster (nearest to centroid).

****Labelling & export****

- Researcher can name clusters and tag individual segments with free-text labels.
- One-click export: CSV of segments with cluster assignment + researcher labels, suitable for use in a future human-rating study.
- Save/load full sessions (data + settings + labels).

4. Anticipated v2 (out of scope now — do not preclude architecturally)

A later phase will analyse ****temporal dynamics****: how conversations move between clusters over time (transition matrices, trajectories, recurrence patterns). Nothing needs to be built for this in v1, but segment ordering and conversation identity must be preserved in the data model so sequence analysis can be added without a rebuild.

5. Verification controls (required in v1)

These exist so the researcher can tell the difference between genuine structure in the data and structure the tools produce by default. Each must be a one-click operation with a plain-English readout.

1. ****Shuffle baseline.**** Re-run the identical pipeline on a shuffled/permutated version of the data (method proposed by engineer — e.g. sentence-order or word-order permutation) and display the result side-by-side with the real run.

2. ****Stability check.**** Re-run clustering across multiple random seeds / bootstrap samples and report agreement (e.g. Adjusted Rand Index), with a simple verdict line such as "cluster assignments are highly stable / moderately stable / unstable across runs."
3. ****Cluster-count diagnostics.**** Show quality metrics (e.g. silhouette scores) across a range of k, so the researcher can see whether any particular number of clusters is genuinely preferred by the data.

6. Explicitly out of scope

- Testing, confirming, or refuting the FRS hypothesis (researcher's job).
- Any model training or fine-tuning.
- Real-time / live LLM integration.
- Topological data analysis (TDA) memory experiments — a separate future project.
- Cloud deployment, user accounts, multi-user features.

7. Deliverables

1. Code repository with clear structure and setup instructions.
2. The workbench application, runnable locally on the researcher's machine (framework of the engineer's choice — e.g. Streamlit or similar — optimised for simplicity of operation).
3. Plain-English user guide covering: loading data, adjusting settings, running verification controls, labelling, exporting, saving sessions.
4. Short recorded walkthrough (screen recording) plus one live handover call.
5. Brief technical notes documenting choices made (embedding model, defaults, known limitations).

8. Acceptance criteria

- The researcher can go from "raw transcript file" to "labelled, exported clusters" without writing or editing any code.
- Verification controls run at one click and produce interpretable output.
- The shuffle baseline visibly degrades structure relative to real data runs (i.e. the control demonstrably works as a control).
- Everything runs locally; no personal transcript data leaves the machine without agreement.

9. Division of responsibilities

****Researcher owns:**** field definitions and theory; interpretation of clusters; all labelling; choice of which conversations/corpora to analyse; final say on any use of hosted services.

****Engineer owns:**** all technical choices within this brief (with documented rationale); proposing defaults; flagging trade-offs in plain English before implementing them.

10. Questions for the engineer to address in their proposal

1. Recommended embedding model and why (with a local-only option).
2. Recommended app framework and what the interface will roughly look like.
3. Proposed default segmentation unit and window size, with reasoning.
4. Proposed shuffle-baseline method.
5. How long transcripts / large corpora will be handled on a standard laptop (performance expectations).
6. Estimated timeline and cost, broken into the milestones below (or a proposed alternative structure).

11. Suggested milestones

- ****M1 — Pipeline working end-to-end**** (command-line acceptable): ingest → embed → reduce → cluster → static plot, on one sample transcript and the AML corpus.
- ****M2 — Interactive workbench****: full exploration, labelling, export, save/load.
- ****M3 — Verification controls + handover****: shuffle baseline, stability check, cluster-count diagnostics, documentation, walkthrough, handover call.